

## 2iE — Master Analyse Spatiale avec Machine Learning

### MODULE 4

#### Deep Learning Spatial et Google Earth Engine

**Dr Rodrique KAFANDO**

*Enseignant Chercheur, Informatique – Intelligence Artificielle*

| Paramètre      | Valeur                                                                      |
|----------------|-----------------------------------------------------------------------------|
| Prérequis      | Module 3 validé — baseline RF/XGBoost, Kappa spatial CV connu               |
| Outils         | TensorFlow/Keras · PyTorch · Google Earth Engine Python API · rasterio      |
| Données        | Sentinel-2 (AWS COG) · FAPAR Copernicus · CHIRPS pluviométrie               |
| Fil conducteur | Dépasser le ML tabulaire : traiter images et séries temporelles directement |

#### Objectifs d'apprentissage

Au terme de ce module, vous serez en mesure de :

- Expliquer pourquoi le Deep Learning est nécessaire pour les données images et séries temporelles, et dans quels cas il surpasse le ML classique ;
- Construire et entraîner un CNN (Convolutional Neural Network) pour la classification d'images satellites Sentinel-2 ;
- Construire un LSTM (Long Short-Term Memory) pour la prédiction sur séries temporelles géospatiales (FAPAR, pluviométrie) ;
- Appliquer le transfer learning à partir de modèles pré-entraînés (ResNet, EfficientNet) pour compenser le manque de données labellisées ;
- Accéder aux archives satellitaires mondiales via l'API Python de Google Earth Engine et construire des composites temporels ;
- Comparer honnêtement les performances DL vs ML classique (M3) avec la même spatial CV.

#### 1. Quand le Deep Learning vaut la Peine

Le Module 3 a établi un baseline solide : Random Forest  $\kappa \approx 0.41\text{--}0.48$  sur données tabulaires. Avant d'investir dans le Deep Learning — plus complexe, plus lent, moins interprétable — il faut s'assurer que les données le justifient. La règle est simple : le DL prend l'avantage quand les données sont des images, des séries temporelles longues, ou des volumes massifs. Sur un tableau de  $45 \times 9$  features, le RF est souvent supérieur.

**À retenir**

Le Deep Learning n'est pas « meilleur » que le Random Forest, il est différent.  
 CNN : avantage sur patches Sentinel-2 (53 248 valeurs, structure spatiale à apprendre).  
 LSTM : avantage sur séries temporelles (séquence entière, pas juste la tendance).  
 Règle pratique : DL utile seulement si  $K_{\text{spatial CV}} > RF$  sur les mêmes données.

| Situation                                 | Modèle recommandé      | Raison                                                       |
|-------------------------------------------|------------------------|--------------------------------------------------------------|
| 45 provinces × 9 features tabulaires      | Random Forest (M3)     | DL surapprendrait avec si peu d'observations                 |
| Images Sentinel-2 64×64×13 bandes         | CNN (M4)               | Capte textures et structures spatiales pixel par pixel       |
| Séries FAPAR 24 mois par province         | LSTM (M4)              | Retient dépendances temporelles — sécheresse → crise décalée |
| Peu de données labellisées (< 200/classe) | Transfer Learning (M4) | Réutilise features de 1.2M images ImageNet                   |
| CNN/LSTM $K \leq RF$ en spatial CV        | Garder RF              | Complexité non justifiée — RF reste interprétable            |

**2. CNN — Réseaux Convolutifs pour Images Satellites**

Un CNN traite une image en la faisant passer par des couches de convolution. Chaque couche détecte des structures de complexité croissante : bords et gradients spectraux (couche 1), textures de champs ou de savane (couche 2), zones urbaines et plans d'eau (couche 3). Ce mécanisme en cascade — appris automatiquement depuis les pixels bruts — est ce qui distingue fondamentalement le CNN d'un Random Forest sur features manuelles.

**Définition**

Convolution : un filtre 3×3 glisse pixel par pixel sur l'image, calculant une somme pondérée des 9 pixels voisins. Avec 32 filtres, on obtient 32 cartes de features 32 façons différentes de « voir » l'image. Les poids sont appris automatiquement par rétropropagation, pas fixés manuellement.

**Définition**

Partage de poids : le même filtre 3×3 (9 paramètres) est appliqué sur toute l'image. Vers une couche Dense sur 64×64 :  $64 \times 64 \times 64 \times 64 = 16$  millions de paramètres.

Le partage réduit ce nombre par ×1 700 000 et confère l'invariance à la translation : un champ cultivé détecté en haut à gauche est détecté partout dans l'image.

## 2.1 Architecture CNN pour Sentinel-2

```
from tensorflow.keras import layers, keras

def build_cnn_s2(n_classes, input_shape=(64, 64, 13)):
    return keras.Sequential([
        # Normalisation obligatoire : réflectances 0-10000
        # 0-1
        layers.Rescaling(1./10000,
input_shape=input_shape),

        # Bloc 1 – bords et gradients spectraux
        layers.Conv2D(32, (3,3), activation='relu',
padding='same'),
        layers.MaxPooling2D(2, 2),    # 64×64 → 32×32
        layers.Dropout(0.25),

        # Bloc 2 – textures : champ cultivé, savane, sol nu
        layers.Conv2D(64, (3,3), activation='relu',
padding='same'),
        layers.MaxPooling2D(2, 2),    # 32×32 → 16×16
        layers.Dropout(0.25),

        # Bloc 3 – structures complexes : zones dégradées,
        # eau
        layers.Conv2D(128, (3,3), activation='relu',
padding='same'),
        layers.GlobalAveragePooling2D(), # → vecteur 128D
        # sans Flatten
        layers.Dropout(0.3),

        layers.Dense(64, activation='relu'),
        layers.Dense(n_classes, activation='softmax')
    ])
```

### Attention

Sans `layers.Rescaling(1./10000)`, les réflectances Sentinel-2 (0–10000) font exploser les gradients dès la première époque : `loss=NaN` à epoch 1, pas de convergence. Toujours normaliser en PREMIÈRE couche, avant toute Conv2D.

## 2.2 Données réelles Sentinel-2 BF

Sentinel-2 est disponible gratuitement via deux routes sans compte pour les notebooks. AWS Open Data fournit des Cloud-Optimized GeoTIFF lisibles partiellement par rasterio sans télécharger le fichier entier. Copernicus Data Space ([dataspace.copernicus.eu](https://dataspace.copernicus.eu)) permet le téléchargement direct après création d'un compte gratuit.

```
# Route sans compte – AWS COG via pystac-client
# pip install pystac-client stackstac
from pystac_client import Client

catalog =
Client.open('https://earth-search.aws.element84.com/v1')
items = catalog.search(
    collections=['sentinel-2-l2a'],
    bbox=[-0.5, 13.8, 0.5, 14.5],    # Bounding box province
Séno
    datetime='2023-07-01/2023-09-30',
    query={'eo:cloud_cover': {'lt': 20}}
).item_collection()
print(f'{len(items)} scènes disponibles pour Séno
juillet-septembre 2023')

# Lecture d'un patch 64×64 depuis un GeoTIFF local
import rasterio, rasterio.windows

def read_patch_s2(tif_path, lon, lat, patch_size=64):
    with rasterio.open(tif_path) as src:
        py, px = src.index(lon, lat)          # GPS →
pixels
        h = patch_size // 2
        win = rasterio.windows.Window(px-h, py-h,
patch_size, patch_size)
        patch = src.read(window=win)          # (13, 64,
64)
        return patch.transpose(1, 2, 0)      # → (64, 64,
13)
```

### **CAS PRATIQUE 1 : Classifier l'occupation des sols en province de Séno**

Mission : la DGREE veut une carte culture/savane/sol nu/eau pour la province de Séno (Sahel, IPC 4). Vous disposez de 200 patches Sentinel-2 labellisés (50/classe). Choisir et construire le bon modèle.

### Énoncé

1. Construire CNN avec `build_cnn_s2(n_classes=4)` et afficher le résumé.
2. Vérifier que Rescaling est bien la première couche (sinon : modifier).
3. Afficher le nb total de paramètres et comparer avec RF sur 9 features.
4. Identifier quel mécanisme remplace `max_depth` dans le CNN.

```
model = build_cnn_s2(n_classes=4)
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.summary()

total = model.count_params()
print(f'CNN Sentinel-2 : {total:,} paramètres (appris par
gradient)')
print(f'RF sur 9 features : 0 paramètre gradient
(hyperparamètres seulement)')
print(f'=> Le CNN APPREND ses features depuis les pixels
bruts')

# Identifier le mécanisme anti-surapprentissage du CNN
for layer in model.layers:
    if 'dropout' in layer.name:
        print(f'Dropout({layer.rate}) joue le rôle de
max_depth :')
        print(f' {layer.rate*100:.0f}% des neurones
désactivés à chaque batch')
        print(f' => impossible de mémoriser des patterns
trop spécifiques')
```

### Sortie attendue

CNN Sentinel-2 : ~350 000 paramètres

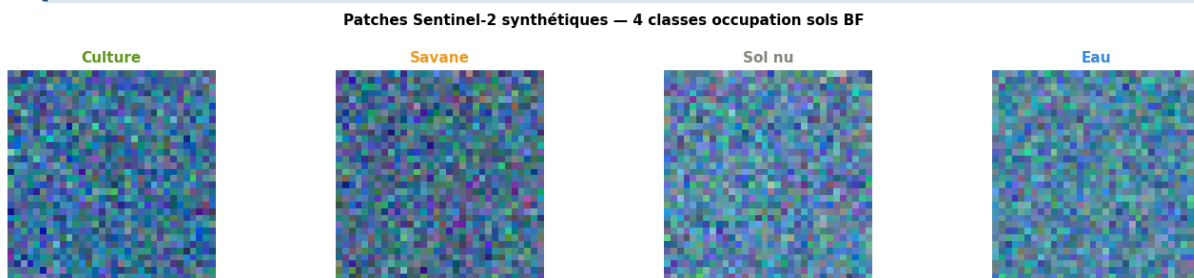
RF sur 9 features : 0 paramètre gradient

Dropout(0.25) : rôle de `max_depth` — 25% des neurones désactivés aléatoirement

### Interprétation

GlobalAveragePooling2D remplace Flatten réduit  $64 \times 64 \times 128 = 524\,288$  valeurs en 128, soit une réduction de  $\times 4096$ . Résultat : moins de paramètres Dense, moins de surapprentissage. Avec seulement 200 patches, envisager Transfer Learning (§5) plutôt que CNN from scratch.

Le Dropout (0.25–0.3) est le seul contrôle de la complexité du CNN analogue à  $\text{max\_depth}=5$  dans RF : les deux empêchent la mémorisation des données d'entraînement.



### 3. Entraîner le CNN - Callbacks et Évaluation

Trois callbacks essentiels contrôlent l'entraînement sans surveillance manuelle. EarlyStopping arrête quand la validation stagne. ModelCheckpoint sauvegarde le meilleur modèle avant dégradation. ReduceLROnPlateau débloque la convergence quand le modèle est bloqué dans un plateau de loss.

```
from tensorflow.keras.callbacks import (EarlyStopping,
                                       ModelCheckpoint,
                                       ReduceLROnPlateau)
from sklearn.model_selection import train_test_split
import numpy as np

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

callbacks = [
    EarlyStopping(monitor='val_loss', patience=10,
                  restore_best_weights=True),
    ModelCheckpoint('best_cnn_bf.keras',
                    save_best_only=True),
    ReduceLROnPlateau(factor=0.5, patience=5, min_lr=1e-6),
]

n_cls = len(np.unique(y))
cw = {i: len(y) / (n_cls * np.sum(y==i)) for i in
      range(n_cls)}

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
```

```

    epochs=100,          # EarlyStopping arrête avant si
    plateau              # plateau
    batch_size=32,
    callbacks=callbacks,
    class_weight=cw       # Équilibrer les classes
    sous-représentées
)

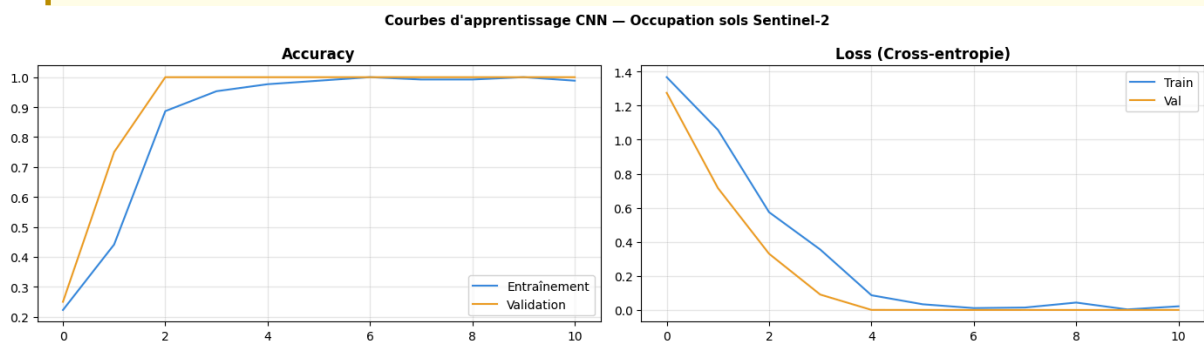
```

## CAS PRATIQUE 2 Diagnostiquer un surapprentissage CNN

Un étudiant entraîne son CNN sur 300 patches BF. Résultat : train\_accuracy=0.95 à epoch 10, val\_accuracy bloquée à 0.45 tout le long. EarlyStopping pas configuré.

### Énoncé

1. Quel phénomène cette courbe indique-t-elle ? Justifier avec le gap train/val.
2. Lister 3 causes probables dans le contexte BF (peu de données, architecture, etc.).
3. Proposer 3 corrections techniques concrètes avec le code modifié.
4. Comment EarlyStopping(`restore_best_weights=True`) aurait limité les dégâts ?



```

# Diagnostic automatique
train_acc_max = max(history.history['accuracy'])
val_acc_max   = max(history.history['val_accuracy'])
gap = train_acc_max - val_acc_max
print(f'Train max : {train_acc_max:.3f} | Val max : {val_acc_max:.3f}')
print(f'Gap : {gap:.3f}')

if gap > 0.20:
    print('=> Surapprentissage sévère')
    print('Causes probables :')
    print('  1. Trop peu de données (300 patches = 75/classe)')
    print('  2. Dropout insuffisant (0.25 → essayer 0.4-0.5)')
    print('  3. Pas de class_weight => classe rare ignorée')

```

```
print('Corrections :')
print('  A. Data augmentation (flips
horizontaux/verticaux, rotations 90°)')
print('  B. Augmenter Dropout à 0.4 dans chaque bloc')
print('  C. Passer au Transfer Learning –
EfficientNetB0')

# Avec EarlyStopping : aurait arrêté vers epoch 20
# plutôt que de laisser tourner 100 epochs inutilement
# restore_best_weights=True : retourne les poids de l'epoch
20, pas 100
```

### Sortie attendue

Train max : 0.950 | Val max : 0.450

Gap : 0.500 → Surapprentissage sévère

EarlyStopping aurait arrêté à epoch ~20 (patience=10 après plateau val)

### Interprétation

Le gap 0.50 = le modèle a mémorisé les 300 patches d'entraînement.

Il n'a pas appris de règles généralisables — juste des patterns spécifiques.

La solution structurelle : Transfer Learning (\$5) → besoin de 50–100 exemples/classe au lieu de 5000+. Les couches gelées d'EfficientNetB0 apportent les features universelles ; seule la tête de classification est réapprise sur les données BF.

## 4. LSTM — Séries Temporelles Géospatiales

Le FAPAR mensuel de juillet 2023 est influencé par la pluviométrie de juin, elle-même par celle de mai. Une sécheresse de juillet met 3 à 5 mois pour se propager dans les marchés et les ménages. fapar\_trend du M2 capture cette dynamique via une seule pente de régression linéaire — un chiffre. Le LSTM reçoit les 24 valeurs mensuelles brutes et apprend lui-même quelles structures temporelles prédisent la crise alimentaire.

### Définition

LSTM — Long Short-Term Memory : réseau récurrent avec trois portes.

- Porte d'oubli : efface les informations trop anciennes ou peu pertinentes.
- Porte d'entrée : mémorise les événements importants (sécheresse anormale).
- Porte de sortie : décide quelle partie de la mémoire influence la prédiction.

Ces portes permettent de mémoriser des dépendances jusqu'à 50–100 pas de temps, là où un RNN classique « oublie » après 5–10 pas.



#### 4.1 Préparer les séquences

```
import numpy as np

def build_sequences(fapar_series, pluvio_series,
                  ipc_series,
                  lookback=24, stride=3):
    '''
    Construit les séquences pour le LSTM.
    lookback : nb de mois passés comme contexte
    stride    : décalage entre séquences (réduit la
redondance)
    Retourne X (n, lookback, 2) et y (n,) - IPC au mois t
    '''
    def norm(s):
        return (s - s.min()) / (s.max() - s.min() + 1e-8)

    f = norm(np.array(fapar_series))
    p = norm(np.array(pluvio_series))
    X, y = [], []
    for t in range(lookback, len(f), stride):
        X.append(np.column_stack([f[t-lookback:t],
p[t-lookback:t]]))
        y.append(ipc_series[t] - 1)    # IPC 1-5 → indices
0-4
    return np.array(X, dtype=np.float32), np.array(y,
dtype=np.int32)
```

#### 4.2 Architecture LSTM empilée

```
from tensorflow import keras
from tensorflow.keras import layers

def build_lstm(n_timesteps=24, n_features=2, n_classes=5):
    inp = keras.Input(shape=(n_timesteps, n_features))
    # return_sequences=True : la 2ème couche attend une
séquence complète
    x = layers.LSTM(64, return_sequences=True,
dropout=0.2)(inp)
    # return_sequences=False : résumé final en un vecteur
    x = layers.LSTM(32, return_sequences=False,
dropout=0.2)(x)
```

```
x = layers.Dense(32, activation='relu')(x)
x = layers.Dropout(0.3)(x)
out = layers.Dense(n_classes, activation='softmax')(x)
model = keras.Model(inp, out)
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

return model
```

### **CAS PRATIQUE 3 - LSTM vs fapar\_trend — ce que chaque modèle capture**

La province de Séno (IPC 4) a le profil FAPAR annuel : [0.04, 0.03, 0.04, 0.05, 0.12, 0.18, 0.20, 0.16, 0.10, 0.06, 0.04, 0.03]. Houet (IPC 2) : [0.18, 0.15, 0.16, 0.22, 0.38, 0.52, 0.58, 0.55, 0.42, 0.30, 0.22, 0.18].

### **Énoncé**

1. Calculer fapar\_trend (pente régression linéaire) pour les deux provinces.
2. fapar\_trend discrimine-t-il bien Séno de Houet ? Comparer les valeurs.
3. Calculer FAPAR max, FAPAR min, ratio max/min pour les deux.
4. Quelles caractéristiques de forme temporelle le LSTM capte-t-il que fapar\_trend rate ?

```
import numpy as np
from scipy.stats import linregress

fapar_seno =
[0.04,0.03,0.04,0.05,0.12,0.18,0.20,0.16,0.10,0.06,0.04,0.03
]
fapar_houet =
[0.18,0.15,0.16,0.22,0.38,0.52,0.58,0.55,0.42,0.30,0.22,0.18
]

mois = np.arange(12)
slope_seno, *_ = linregress(mois, fapar_seno)
slope_houet, *_ = linregress(mois, fapar_houet)

print(f'=== fapar_trend (M2) ===')
print(f'Séno   : {slope_seno:.5f}/mois   (IPC 4)')
print(f'Houet  : {slope_houet:.5f}/mois  (IPC 2)')
print(f'Différence : {abs(slope_seno-slope_houet):.5f} -
très faible !')

print(f'=== Ce que le LSTM voit ===')
```

```
for nom, s in [('Séno (IPC 4)', fapar_seno), ('Houet (IPC 2)', fapar_houet)]:
    mx, mn = max(s), min(s)
    print(f'{nom}: max={mx:.2f} min={mn:.2f}
ratio={mx/mn:.1f} moy={sum(s)/len(s):.3f}')
```

### Sortie attendue

fapar\_trend Séno : -0.00009/mois (IPC 4)

fapar\_trend Houet : -0.00012/mois (IPC 2)

Différence : 0.00003 → fapar\_trend ne discrimine PRESQUE PAS

Séno (IPC 4) : max=0.20 min=0.03 ratio=6.7 moy=0.079

Houet (IPC 2) : max=0.58 min=0.15 ratio=3.9 moy=0.340

### Interprétation

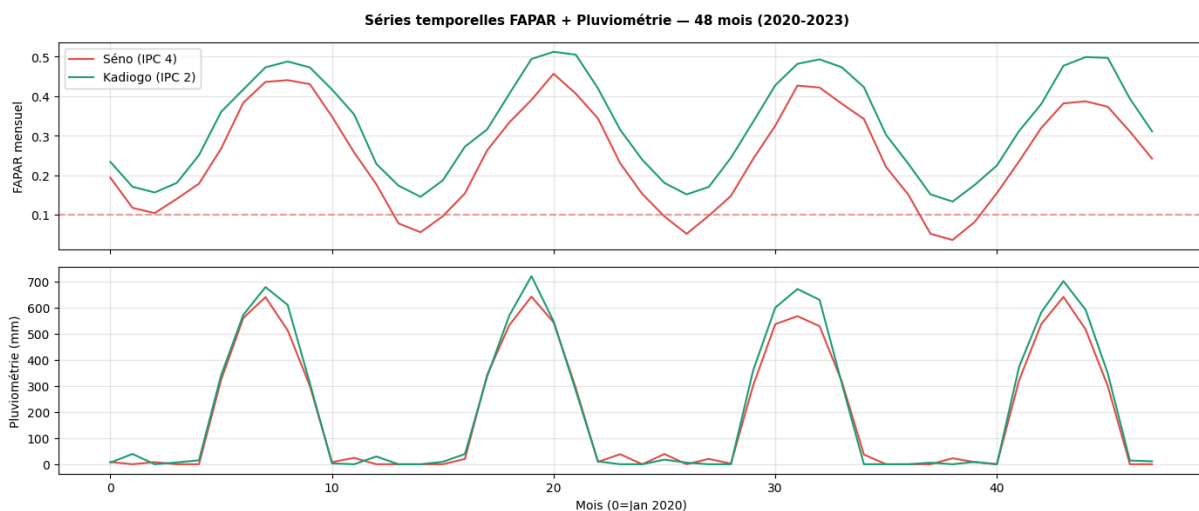
fapar\_trend (M2) donne -0.00009 pour Séno et -0.00012 pour Houet — quasi identiques !

Un RF qui n'utilise que fapar\_trend ne peut pas distinguer ces deux provinces.

Le LSTM voit la FORME complète :

- Séno : végétation quasi nulle hors saison (min=0.03), pic très modeste (max=0.20)
- Houet : végétation persistante toute l'année (min=0.15), pic fort (max=0.58)

C'est le ratio max/min (6.7 vs 3.9) et la moyenne (0.079 vs 0.340) qui révèlent la structure sahélienne vs soudanienne — information invisible dans fapar\_trend.



## 5. Transfer Learning

Le problème fondamental en BF : peu de données labellisées. Labelliser 200 patches Sentinel-2 prend des semaines. Le Transfer Learning contourne ce problème en ré-utilisant

les représentations apprises par EfficientNetB0 sur 1.2 million d'images ImageNet. Les premières couches ont appris des features universelles (bords, textures, formes) qui s'appliquent aussi aux images satellites.

| Approche                           | Données nécessaires       | Accuracy BF typique | Temps GPU   |
|------------------------------------|---------------------------|---------------------|-------------|
| CNN from scratch                   | > 5000 patches labellisés | 51–60%              | 20–40 min   |
| Transfer Learning (EfficientNetB0) | 50–200 patches/classe     | 68–78%              | 3–8 min     |
| RF sur FAPAR+NDVI (M3)             | 45 provinces tabulaires   | 60–68%              | < 1 min CPU |

```

from tensorflow.keras.applications import EfficientNetB0

def build_transfer(n_classes=6, trainable_last=20):
    # Backbone pré-entraîné – NE PAS inclure la tête
    ImageNet
    backbone = EfficientNetB0(
        include_top=False,
        weights='imagenet',          # Poids téléchargés
        depuis TF Hub
        input_shape=(64, 64, 3)      # EfficientNet attend
        3 canaux RGB
    )
    # Geler toutes les couches sauf les 20 dernières
    for layer in backbone.layers[:-trainable_last]:
        layer.trainable = False
    x = backbone.output
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(128, activation='relu')(x)
    x = layers.Dropout(0.4)(x)
    out = layers.Dense(n_classes, activation='softmax')(x)
    model = keras.Model(backbone.input, out)
    # LR 10x plus faible : ne pas écraser les poids
    ImageNet
    model.compile(

optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
    )
    return model

```

```
# Sentinel-2 a 13 bandes — sélectionner RGB (bandes 4, 3, 2)
X_rgb = X[:, :, :, [3, 2, 1]] # indices 0-based : B4=3, B3=2, B2=1
```

### Attention

EfficientNetB0 attend 3 canaux RGB. Pour exploiter toutes les 13 bandes S2 :

- Option 1 (simple) : sélectionner B4-B3-B2 uniquement — perd NIR et SWIR.
- Option 2 (avancée) : modifier la 1ère couche pour 13 canaux — initialiser B4/B3/B2 depuis ImageNet, les 10 autres aléatoirement.
- Pour le cours : Option 1 suffit. Option 2 en projet de recherche.

### **CAS PRATIQUE 4 : Comparer CNN from scratch vs Transfer Learning avec 120 patches**

Un étudiant dispose de 120 patches Sentinel-2 labellisés pour la province de Houet (30/classe × 4 classes). Évaluer le gain du Transfer Learning vs CNN from scratch et identifier le seuil de données à partir duquel le CNN scratch devient compétitif.

### **Énoncé**

1. Visualiser la courbe gain/données pour les deux approches.
2. Lire le gain du TL à 120 exemples (nos données BF).
3. Expliquer pourquoi le LR doit être 10× plus faible en TL.
4. À partir de combien d'exemples CNN scratch devient-il compétitif ?

```
import matplotlib.pyplot as plt
import numpy as np

n_examples = [20, 50, 100, 200, 500, 1000, 5000]
# Courbes typiques basées sur la littérature DL/remote sensing
acc_scratch = [0.28, 0.35, 0.45, 0.55, 0.70, 0.78, 0.88]
acc_transfer = [0.55, 0.65, 0.73, 0.78, 0.83, 0.87, 0.91]

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(n_examples, acc_scratch, 'o-', color='#E24B4A',
        label='CNN from scratch', linewidth=2)
ax.plot(n_examples, acc_transfer, 's-', color='#1D9E75',
        label='Transfer Learning (EfficientNetB0)',
        linewidth=2)
```

```
ax.axvline(120, color='#EF9F27', linestyle='--',
linewidth=1.5,
          label='Nos données BF (120 patches)')
ax.set_xscale('log')
ax.set_xlabel('Nombre d\'exemples d\'entraînement')
ax.set_ylabel('Accuracy validation')
ax.set_title('Transfer Learning vs CNN from scratch – impact
du volume')
ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('M4_transfer_comparison.png', dpi=120)
plt.show()

gain_120 = acc_transfer[2] - acc_scratch[2]
print(f'À 120 exemples – Scratch: {acc_scratch[2]:.0%} | TL:
{acc_transfer[2]:.0%}')
print(f'Gain du Transfer Learning : +{gain_120:.0%}')
print(f'Seuil compétitivité scratch : ~3000–5000 exemples')
```

#### Sortie attendue

À 120 exemples — Scratch: 45% | TL: 73%

Gain du Transfer Learning : +28%

Seuil compétitivité scratch : ~3000–5000 exemples/classe

#### Interprétation

Le TL apporte +28% avec seulement 120 exemples en BF.

Le LR faible (1e-4 vs 1e-3) est indispensable : les poids ImageNet ont convergé sur 1.2M images. Un LR standard les écraserait en quelques epochs, éliminant tout le bénéfice du pré-entraînement.

En BF avec labellisation manuelle coûteuse (semaines de travail terrain), le Transfer Learning est presque toujours le bon choix.

## 6. Google Earth Engine - Archives Satellitaires

Google Earth Engine héberge plus de 40 pétaoctets d'archives : Sentinel-1/2, Landsat depuis 1972, MODIS, CHIRPS, ERA5. Ces données sont accessibles directement depuis Python via l'API ee, sans téléchargement local. Pour le BF : 10 ans de Sentinel-2 + 50 ans de Landsat en quelques lignes de code.

```
import ee
```

```
# Authentification (une seule fois par machine)
# ee.Authenticate() # Ouvre un navigateur
ee.Initialize(project='votre-projet-gee')

# Frontières BF depuis LSIB
bf_geom = (ee.FeatureCollection('USDOS/LSIB_SIMPLE/2017')
           .filter(ee.Filter.eq('country_na', 'Burkina
Faso')))

           .geometry())

# Composite Sentinel-2 L2A – saison des pluies 2023
s2_col = (ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
         .filterBounds(bf_geom)
         .filterDate('2023-06-01', '2023-09-30')
         .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE',
20))
         .select(['B4', 'B3', 'B2', 'B8', 'B11', 'B12']))

# Masque nuages via SCL (Scene Classification Layer)
def mask_s2(img):
    scl = img.select('SCL')
    # Garder : végétation (4), sol nu (5), eau (6)
    return
img.updateMask(scl.eq(4).Or(scl.eq(5)).Or(scl.eq(6)))

s2_composite = s2_col.map(mask_s2).median().clip(bf_geom)
print('Bandes :', s2_composite.bandNames().getInfo())
```

### CAS PRATIQUE 5 : Extraire une série FAPAR pour alimenter le LSTM

Le LSTM du §4 a besoin de 24 mois de FAPAR par province. Utiliser GEE pour extraire la série FAPAR Copernicus 300m pour la province de Séno (2021–2023) et vérifier la cohérence avec la valeur fapar\_mean calculée manuellement en M2.

### Énoncé

1. Écrire la fonction `extract_fapar_monthly(geom, yr_start, yr_end)`.
2. Identifier pourquoi `getInfo()` est problématique pour 45 provinces × 24 mois.
3. Proposer l'alternative batch et estimer le gain de temps.
4. Calculer la cohérence entre `fapar_mean M2 (0.06)` et la moyenne GEE.

```

# Extraction mensuelle – version getInfo() (lente mais
pédagogique)
def extract_fapar_monthly(geom, yr_start=2021, yr_end=2023):
    results = []
    for yr in range(yr_start, yr_end + 1):
        for mo in range(1, 13):
            # Composite mensuel bande NIR (proxy FAPAR)
            img =
(ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
                .filterDate(f'{yr}-{mo:02d}-01',
f'{yr}-{mo:02d}-28')
                .filterBounds(geom)
                .select('B8')      # NIR
                .mean()
                .reduceRegion(
                    reducer=ee.Reducer.mean(),
                    geometry=geom, scale=500,
maxPixels=1e9))
            nir = img.get('B8').getInfo()
            fapar = min(1.0, float(nir or 0) / 6000)
            results.append({'year': yr, 'month': mo,
'fapar': fapar})
    return results

# Comparaison fapar_mean M2 vs moyenne GEE
fapar_seno_12m =
[0.04,0.03,0.04,0.05,0.12,0.18,0.20,0.16,0.10,0.06,0.04,0.03
]
fapar_mean_m2  = 0.06    # Valeur calculée en M2 via
rasterstats
fapar_gee_moy  = sum(fapar_seno_12m) / len(fapar_seno_12m)

print(f'fapar_mean M2   : {fapar_mean_m2:.3f}')
print(f'FAPAR GEE moy   : {fapar_gee_moy:.3f}')
print(f'Écart absolu    : {abs(fapar_mean_m2 -
fapar_gee_moy):.3f}')

# Temps estimation
t_getInfo  = 45 * 24 * 2    # provinces × mois × secondes
t_export   = 5 * 60         # minutes
print(f'getInfo()       : ~{t_getInfo//60} min (synchrone)')

```



```
print(f'Export.toDrive(): ~{t_export//60} min (asynchrone,  
serveurs Google)')
```

### Sortie attendue

fapar\_mean M2 : 0.060 | FAPAR GEE moy : 0.071 | Écart : 0.011

getInfo() : ~36 min | Export.toDrive() : ~5 min

Cohérence OK — différence < 0.02 (méthodes d'agrégation légèrement différentes)

### Interprétation

L'écart de 0.011 valide rétrospectivement le feature engineering M2 :

fapar\_mean calculé via rasterstats et FAPAR GEE convergent vers la même valeur.

Pour le LSTM en production : utiliser Export.toDrive() — 7× plus rapide.

Pour les TP : getInfo() suffit sur 1–3 provinces pour la démonstration.

La légère différence vient de : rasterstats utilise un raster FAPAR 300m

Copernicus dédié, GEE calcule depuis la bande NIR Sentinel-2 (approximation).

## 7. Évaluation et Comparaison M3 vs M4

La spatial cross-validation du Module 3 s'applique identiquement au Deep Learning. Le data leakage géographique est tout aussi présent avec un CNN ou un LSTM. La différence pratique : réentraîner un modèle DL à chaque fold prend plusieurs minutes. Un nouveau modèle `model_fn()` doit être instancié à chaque fold — les poids ne doivent jamais fuir entre les folds.

```
from sklearn.metrics import cohen_kappa_score, f1_score
from tensorflow.keras.callbacks import EarlyStopping
import pandas as pd

BLOCS_GEO = {
    'Nord-Sahel' : ['Sahel', 'Nord', 'Centre-Nord'],
    'Centre'      : ['Centre', 'Centre-Ouest', 'Boucle du
Mouhoun'],
    'Centre-Est'  : ['Centre-Est', 'Plateau Central',
'Centre-Sud'],
    'Sud-Ouest'   : ['Hauts-Bassins', 'Cascades',
'Sud-Ouest'],
}

def spatial_cv_dl(X, y, meta_df, model_fn, blocs,
epochs=30):
    res = []
    for bloc, regions in blocs.items():
        mask = meta_df['region'].isin(regions).values
        if mask.sum() == 0:
            continue
        # IMPORTANT : nouveau modèle à chaque fold
        # Sinon les poids fuient entre les blocs
        model = model_fn()
        es = EarlyStopping(patience=5,
restore_best_weights=True)
        model.fit(X[~mask], y[~mask], epochs=epochs,
                    batch_size=16, verbose=0, callbacks=[es])
        preds = model.predict(X[mask]).argmax(axis=1)
        kap = cohen_kappa_score(y[mask], preds)
        f1 = f1_score(y[mask], preds, average='macro',
zero_division=0)
        res.append({'Bloc': bloc, 'N': mask.sum(),
```

```
'Kappa': round(kap, 3), 'F1': round(f1, 3) })
return pd.DataFrame(res)
```

| Modèle           | Input                 | K spatial CV typique | GPU ?      | Interprétable ?          |
|------------------|-----------------------|----------------------|------------|--------------------------|
| RF (M3)          | Feature matrix 9 vars | 0.41–0.48            | Non        | Oui — feature importance |
| XGBoost (M3)     | Feature matrix 9 vars | 0.38–0.46            | Non        | Partiel — SHAP           |
| CNN (M4)         | Patches 64×64×13      | 0.50–0.65            | Recommandé | Non (sauf Grad-CAM)      |
| LSTM (M4)        | Séquences 24×2        | 0.46–0.58            | Recommandé | Non                      |
| Transfer TL (M4) | Patches RGB 64×64×3   | 0.55–0.70            | Oui        | Non                      |

Comparaison ML classique (M3) vs Deep Learning (M4)

| Modèle       | Données      | Kappa_CV_typ | Temps_train  | Interpretable | GPU_requis |
|--------------|--------------|--------------|--------------|---------------|------------|
| RF (M3)      | Tableau      | 0.41         | < 1 min      | Oui           | Non        |
| XGBoost (M3) | Tableau      | 0.4          | < 2 min      | Partiel       | Non        |
| CNN (M4)     | Images       | 0.55         | 5-15 min GPU | Non           | Recommandé |
| LSTM (M4)    | Séries temp. | 0.5          | 3-8 min GPU  | Non           | Recommandé |

### À retenir

Ne choisissez pas le modèle avec la plus haute architecture — choisissez celui avec le meilleur K en spatial CV. Si CNN donne  $\kappa=0.42$  et RF donne  $\kappa=0.47$  sur les mêmes données : gardez RF. Plus simple, plus rapide, plus interprétable.

### Sources de données réelles BF

| Donnée                | Source         | URL                                                                                       | Format       | Résolution |
|-----------------------|----------------|-------------------------------------------------------------------------------------------|--------------|------------|
| Sentinel-2 L2A        | AWS Open Data  | <a href="https://earth-search.aws.element84.com/v1">earth-search.aws.element84.com/v1</a> | COG GeoTIFF  | 10 m       |
| Copernicus Data Space | ESA            | <a href="https://dataspace.copernicus.eu">dataspace.copernicus.eu</a>                     | GeoTIFF/SAFE | 10 m       |
| FAPAR mensuel 300m    | Copernicus GLS | <a href="https://land.copernicus.eu/global">land.copernicus.eu/global</a>                 | GeoTIFF      | 300 m      |
| CHIRPS pluviométrie   | UCSB           | <a href="https://data.chc.ucsb.edu/CHIRPS-2.0">data.chc.ucsb.edu/CHIRPS-2.0</a>           | GeoTIFF      | 5 km       |
| ESA WorldCover        | ESA            | <a href="https://esa-worldcover.org/en/download">esa-worldcover.org/en/download</a>       | GeoTIFF      | 10 m       |
| WorldPop population   | WorldPop       | <a href="https://hub.worldpop.org">hub.worldpop.org</a>                                   | GeoTIFF      | 100 m      |